
Communication-Efficient SGD: Sync, Local, and Async Training Under System Constraints

Lisa Li* Jacqueline Wen*

Abstract

Communication overhead is a central bottleneck in distributed deep learning, and different synchronization strategies result in fundamentally different tradeoffs between communication cost, convergence, and resilience to system faults. We implement and empirically evaluate four distributed SGD variants – synchronous all-reduce SGD, local SGD with periodic parameter averaging, and asynchronous SGD with a shared-memory parameter server, compared against a single-worker baseline – on CIFAR-10 across three axes: communication efficiency, system stressor resilience, and worker scaling. Our results reveal an asymmetry between uniform per-sync communication latency and skewed per-batch worker slowdowns (stragglers): local SGD is nearly 19× more resilient to latency due to its reduced synchronization frequency, yet both methods incur equivalent total wall-clock overhead under stragglers. Sweeping the averaging interval from $H = 1$ to $H = 32$ reduces communication by 96% with accuracy change indistinguishable from training noise in our setting, and scaling experiments show local SGD achieves consistently higher wall-clock efficiency than synchronous SGD, suggesting that infrequent synchronization is advantageous across multiple systems dimensions for moderate-scale distributed training.

1. Background

Large-scale machine learning training is often limited not only by computation, but also by the cost of communication between distributed workers. Standard synchronous data-parallel SGD communicates every step and typically provides stable optimization, but it can become bottlenecked by network delay, limited bandwidth, and slow workers. Communication-efficient alternatives such as local SGD reduce synchronization frequency by allowing workers to take multiple local steps before averaging, and asynchronous methods remove strict synchronization entirely at the cost of allowing stale gradients. Prior works suggest that local

SGD can reduce communication substantially while preserving good convergence in many settings, and later works have explored ways to recover accuracy in communication-efficient regimes to improve robustness to delay and varying network topologies.

2. Previous Works

2.1. Synchronous Distributed SGD

In data-parallel distributed training, W workers each process a disjoint partition of the data and synchronize gradients via all-reduce before each parameter update, equivalent to computing the gradient over a combined mini-batch of size $W \cdot |\mathcal{B}|$. A blocking barrier at every step makes synchronous SGD sensitive to communication latency and straggler workers. A further consequence is the large-batch problem: optimal learning rate scales with effective batch size, so distributed accuracy degrades without a corresponding LR increase.

(Goyal et al., 2017) provided a theoretical foundation showing that large-batch training does not necessarily hurt generalization and that high-accuracy models can be trained faster through scaling without loss of performance. They introduced the linear scaling rule, where increasing the learning rate proportional to batch size maintains convergence at scale, along with gradual warmup, where training starts with a lower learning rate and ramps up during the first few epochs to avoid optimization instability from large batches in early training. Their method used synchronous SGD with all-reduce to average gradients across GPUs, reducing ImageNet training time from roughly 29 hours to 1 hour.

2.2. Local SGD

Local SGD (Stich, 2019; Lin et al., 2020) is a variation of traditional SGD where each worker performs several SGD steps independently before periodically synchronizing and averaging model parameters across workers. Unlike standard distributed mini-batch SGD, workers do not communicate after every gradient update, substantially lowering communication overhead in distributed machine learning training. Workers take H independent gradient steps before averaging parameters, reducing synchronization rounds

by a factor of H . The central tension is client drift, in which workers’ models diverge during local steps. (Stich, 2019) provides a theoretical analysis of local SGD, showing convergence rates for convex problems comparable to traditional mini-batch SGD while reducing communication between workers up to a factor of $\sqrt{T}$, where T is the number of training iterations.

2.3. Asynchronous SGD

Asynchronous SGD (Dean et al., 2012) removes barriers entirely via a parameter server: workers independently pull parameters, compute gradients, and push updates without waiting for peers. Although synchronous SGD has strong guarantees of convergence, the entire system runs at the speed of the slowest worker (the “straggler”), which is a significant waste of resources. However, without global synchronization, workers are always working with slightly stale parameters.

Gradients computed with stale parameters introduce noise that can slow or destabilize convergence. However (Agarwal & Duchi, 2011) showed that even with delayed gradients, for a system with n nodes and T iterations, asynchronous SGD still converge at $O(1/\sqrt{nT})$ for smooth convex problems. For any distributed system of n nodes, even with no delays, it optimally converges at $O(1/\sqrt{nT})$. In other words, the impact of the stale gradients are asymptotically negligible.

3. System Design and Implementation

3.1. Overview

We implement four training strategies in PyTorch: a single-worker baseline, synchronous SGD, local SGD, and asynchronous SGD with a parameter server. Synchronous and local SGD use `torch.distributed` with `mp.spawn`, and asynchronous SGD uses `mp.Process` with shared-memory tensors. All methods use the same three-layer CNN (~ 2.2 M parameters), optimizer (vanilla SGD, $\eta=0.01$), and data partitioning, with the 50,000 samples in the CIFAR-10 training set split into W partitions, one per worker.

3.2. Synchronous and Local SGD

Synchronous SGD all-reduces gradient tensors after every backward pass. Local SGD instead all-reduces model parameters every H steps and at each epoch boundary, which reduces communication by a factor of H . Communication is tracked per synchronization round.

3.3. Asynchronous SGD and Parameter Server

The parameter server holds the global model in shared-memory tensors with a single `mp.Lock`, serializing reads

and writes. Workers loop independently pull parameters, release the lock, compute a gradient, and push an SGD update. Staleness τ is measured between the pull and push steps. We implement three variants: standard async, staleness-aware learning rate scaling ($\eta/(1+\tau)$), and bounded staleness ($\tau_{\max}=4$). We use a locking parameter server more similar to DistBelief (Dean et al., 2012) than to Hogwild! (Recht et al., 2011).

3.4. System Stressor Simulation

We inject two stressors to evaluate resilience. **Communication latency**: a fixed `sleep(t)` after each all-reduce in synchronous and local SGD, modeling uniform network delay per-sync. In this case, async SGD is unaffected because since it performs no collective operations. **Stragglers**: k workers sleep t_s seconds before each mini-batch, reducing their throughput without affecting gradient computation.

4. Experiments

4.1. Setup

All experiments use batch size 64, no momentum, and no learning rate schedule, on CPU only. The CNN consists of two convolutional blocks with max-pooling followed by a two-layer MLP classifier. Unless otherwise stated, distributed runs use $W=4$ workers, and the training set is partitioned into W slices with per-worker shuffling each epoch.

We evaluate three metrics. **Statistical efficiency** is measured as validation accuracy relative to total communication bytes, with communication volume as a proxy for statistical work. **Hardware efficiency** is measured via wall-clock training time and, in the scaling experiment, parallel speedup relative to a single-worker baseline. **Communication cost** is tracked per epoch as $N_{\text{params}} \times 4 \times 2$ bytes per synchronization round, which slightly overestimates ring all-reduce cost ($2(W-1)/W$ of data size) but is exact for our async pulling and pushing model.

As a note on accuracy, each distributed worker receives only $1/W$ of the training data per epoch, and gradient averaging across W workers on batch size 64 produces an effective batch size of $W \times 64 = 256$ without a corresponding learning rate increase. These effects reduce distributed accuracy relative to the single-worker baseline. For this project, absolute accuracy is modest, but comparisons between distributed methods are fair since they are all equally affected.

5. Results

5.1. Experiment 1: Communication Efficiency

We sweep the local SGD averaging interval over $H \in \{1, 2, 4, 8, 16, 32\}$ and compare against synchronous SGD, a single-worker baseline, and the three async SGD variants over 15 epochs. Results are summarized in Table 1 and Figure 1.

Local SGD accuracy-communication tradeoff. The central result is that local SGD’s accuracy-communication tradeoff is nearly flat: increasing H from 1 to 32 reduces communication by 96% (from 51,585 MB to 1,842 MB) while accuracy drops only slightly (53.15% to 51.93%), well within the noise of the training setup. Communication halves exactly with each doubling of H , as expected from the experimental setup. As a correctness check, local SGD with $H=1$ matches synchronous SGD almost exactly (53.15% vs. 53.23%), as both perform a synchronization at every step. The minor difference arises only because local SGD averages parameters after the optimizer step while synchronous SGD averages gradients before it.

Wall-clock time. Wall-clock time decreases modestly with H , with an 11% reduction from $H=1$ (1741 s) to $H=32$ (1547 s). On a single machine with a low-latency backend, communication overhead is a small fraction of per-batch compute time; in a real distributed environment with network latency, this gap would be substantially larger.

Asynchronous SGD variants. Standard async SGD achieves 68.41% accuracy, approaching the single-worker baseline (70.62%) and significantly exceeding synchronous SGD (53.23%). However, this advantage is a throughput effect rather than an algorithmic one. Since async workers are not constrained by a synchronization barrier, the epoch boundary is defined by total global steps rather than per-worker steps (~ 195), meaning async performs approximately $4\times$ more gradient updates per epoch. As a result, we interpret async accuracy in terms of per-update efficiency rather than per-epoch comparison.

Staleness-aware learning rate scaling (async-SA) reaches only 52.81%, making it nearly indistinguishable from synchronous SGD. With $W=4$ workers sharing a single machine, average staleness is naturally bounded at approximately $W-1 = 3$, so the correction divides the learning rate by approximately 4 (from 0.01 to 0.0025), which is too conservative for this task. This is likely indicative of a regime mismatch, as staleness-aware corrections target large-scale deployments with high staleness and are counterproductive when staleness is naturally low.

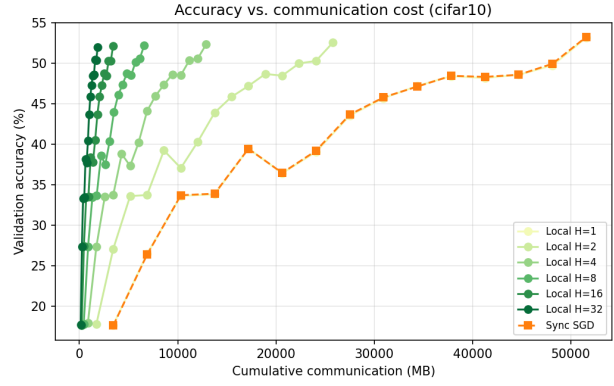


Figure 1. Accuracy vs. cumulative communication bytes on CIFAR-10 (4 workers, 15 epochs) for local SGD with $H \in \{1, 2, 4, 8, 16, 32\}$, sync SGD, and async variants. Communication halves with each doubling of H , and the accuracy cost is 1.2 percentage points across the full range.

5.2. Experiment 2: System Stressors

We evaluate synchronous SGD, local SGD ($H=10$), and async SGD under two controlled stressors over 5 epochs with 4 workers.

Communication latency. We inject a fixed delay t after each all-reduce to simulate uniform per-synchronization network overhead, iterating over $t \in \{0, 0.01, 0.05, 0.1\}$ s. Results are shown in Figure 2. Async SGD incurs no all-reduce overhead and is unaffected by this stressor, but we include it as a reference.

Synchronous SGD incurs substantial overhead that grows linearly with t , closely matching the theoretical prediction of $N_{\text{sync}} \times t \approx 975 \times t$ seconds over 5 epochs. Local SGD ($H=10$) is nearly unaffected: it performs only ~ 100 synchronization rounds over 5 epochs, yielding a theoretical overhead of $\sim 100 \times t$, which is roughly $10\times$ less than synchronous SGD. At $t=0.1$ s, the empirically observed overhead ratio is $18.9\times$ (113.3 s vs. 6.0 s), which directly demonstrates the primary systems advantage of local SGD, with reduced synchronization proportionally reducing latency sensitivity (see Figure 2 in the Appendix).

Straggler workers. We let $k \in \{0, 1, 2\}$ of 4 workers to sleep 0.05 s before each mini-batch, reducing their throughput without affecting gradient computation.

The straggler results, shown in Figure 3, yield a non-obvious result: synchronous and local SGD incur *equivalent* total overhead under straggler injection. For synchronous SGD, the straggler delays every batch at the all-reduce barrier, resulting in total overhead $N_{\text{batches}} \times t_s = 975 \times 0.05 = 48.75$ s, closely matching the observed 48.8 s. For local

Table 1. Experiment 1 results on CIFAR-10 (4 workers, 15 epochs; baseline uses 1 worker). Async variants accumulate $\sim 4\times$ more communication than sync/local because they perform $\sim 4\times$ more gradient updates per epoch.

Method	Acc (%)	Time (s)	Comm (MB)
Baseline (1 worker)	70.62	5122	—
Sync SGD	53.23	1742	51,585
Async SGD	68.41	1298	205,584
Async-SA	52.81	1303	205,549
Async (bounded $\tau=4$)	69.84	1677	205,549
Local $H=1$	53.15	1741	51,585
Local $H=2$	52.53	1648	25,792
Local $H=4$	52.31	1596	12,896
Local $H=8$	52.16	1569	6,580
Local $H=16$	52.10	1555	3,421
Local $H=32$	51.93	1547	1,842

SGD with $H=10$, the straggler delays each of its H local steps before every synchronization point, so the barrier wait per synchronization round is $H \times t_s$, and total overhead is $(N/H) \times H \times t_s = N \times t_s$. This is identical to synchronous SGD, which is further supported by the observed 51.5 s. Unlike latency, where synchronization frequency is the determining factor, straggler overhead is determined by total computation, so the reduced sync frequency of local SGD becomes irrelevant. A second finding is that one straggler and two stragglers produce nearly identical overhead in both synchronous and local SGD (1.3 s and 1.6 s differences, respectively): when multiple slow workers sleep for the same duration, the barrier wait is set by the slowest worker, not the count.

Async SGD shows small, monotonically increasing overhead (12.8 s per straggler), as slow workers contribute fewer gradient updates but do not block faster workers. Detailed results are shown in Figure 3 in the Appendix.

5.3. Experiment 3: Worker Scaling

We measure how wall-clock time and accuracy scale with worker count $W \in \{1, 2, 4\}$ for synchronous and local SGD ($H=10$) over 15 epochs. Speedup is defined as T_1/T_W relative to the single-worker distributed baseline; efficiency is speedup/ W .

Both methods demonstrate sub-linear speedup (Figure 4 in the Appendix), as expected when communication overhead is significant relative to per-batch compute time. Local SGD achieves higher efficiency for every worker count (0.81 vs. 0.77 at $W=2$; 0.53 vs. 0.48 at $W=4$), which is consistent with its lower communication frequency. At $W=4$, synchronous SGD reaches $1.92\times$ speedup while local SGD

reaches $2.12\times$, and the gap between each method and the ideal linear scaling reflects communication overhead.

Accuracy degrades with worker count for both methods, from 70.78% at $W=1$ to $\sim 53\%$ at $W=4$. As noted in the setup, this reflects the effects of each worker processing only $1/W$ of the data per epoch and the unscaled learning rate under a larger effective batch size. However, all methods at a given worker count are equally affected, preserving the validity of comparisons between them. The identical accuracy of single-worker synchronous and local SGD (70.78%) confirms that both reduce to equivalent single-worker training when $W=1$.

6. Discussion

Strengths. A key strength is the controlled experimental design: all methods share the same hyperparameters, model, and data partitioning, so observed differences are attributable to synchronization strategy alone. A second strength is breadth of evaluation: rather than focusing on a single axis, we measure communication efficiency, stressor resilience, and worker scaling together, which reveals interactions such as the latency-straggler asymmetry. Furthermore, implementing three async variants within the same framework enables a clean comparison of staleness-handling strategies under identical conditions.

Weaknesses. A key weakness of this project is that we tried to emulate a distributed network on a single physical machine using shared memory, instead of using a real distributed network. We tried to simulate the network delays as realistically as possible, but communication is conducted via multiprocessing, so the absolute wall-clock differences between methods are shorter than what would actually be observed over a network with real bandwidth and latency constraints. Additionally, our latency stressor is a simplified model that does not have the variable or correlated network delays. Another key weakness is that due to the limitations on compute, we were unable to train CIFAR to extremely high accuracies. This makes it difficult to find subtle accuracy differences between the four training methods. Finally, we could only have a maximum of four workers train our small CNN, and we acknowledge that the results may differ when we scale substantially more.

Impact and Future Work

Impact. The results suggest some practical takeaways. First, local SGD with moderate H is preferable to synchronous SGD when communication is the bottleneck as it reduces the synchronization overhead with little accuracy regression. However, when delays are mainly caused by stragglers, it is preferable to switch to asynchronous SGD. Finally, staleness-aware learning rate corrections should only be

applied when there is high staleness. In our low-staleness training environments, these staleness-aware learning corrects cause convergence regressions.

Future Work. This study could be extended by extending the hardware that our experiments were conducted on. It would be interesting to investigate how these results extend for more workers/in an actual distributed system. Second, incorporating momentum and a learning rate warmup schedule (Goyal et al., 2017) would more accurately represent a state of the art synchronous system. Finally, replacing our locking parameter server with a true lock-free async implementation (Hogwild! (Recht et al., 2011)) would enable a cleaner comparison of the two async systems and test whether lock contention is a meaningful bottleneck at larger worker counts.

7. Conclusion

We implemented and compared four distributed SGD strategies across three experiments on CIFAR-10. Local SGD’s accuracy-communication tradeoff is nearly flat, with a 96% reduction in communication having nearly no effect on accuracy, and its $18.9\times$ lower latency overhead confirms that reduced synchronization directly translates to latency resilience. However, local SGD offers no advantage over synchronous SGD under stragglers, since straggler cost depends on total computation rather than synchronization frequency in this setup, an asymmetry with practical implications for distributed training. Async SGD avoids blocking under both stressors, though its accuracy advantage reflects higher update throughput rather than superior per-update convergence. These findings suggest that the right synchronization strategy depends on which bottleneck dominates: local SGD for latency-constrained settings, async SGD when stragglers are the primary concern.

Acknowledgements

This project was conducted for Cornell’s CS 6787 and was guided by Professor De Sa. We are grateful for the knowledge we gained through lectures and paper readings in this course and the guidance for our project.

References

- Agarwal, A. and Duchi, J. C. Distributed delayed stochastic optimization. In *Advances in Neural Information Processing Systems (NIPS)*, 2011. URL <https://arxiv.org/abs/1104.5525>.
- Dean, J., Corrado, G., Monga, R., Chen, K., Devin, M., Mao, M., Ranzato, M., Senior, A., Tucker, P., Yang, K., Le, Q. V., and Ng, A. Y. Large scale distributed deep networks. In *Advances in Neural Information Processing*

Systems, 2012.

- Goyal, P., Dollár, P., Girshick, R., Noordhuis, P., Wesolowski, L., Kyrola, A., Tulloch, A., Jia, Y., and He, K. Accurate, large minibatch SGD: Training ImageNet in 1 hour. *arXiv preprint arXiv:1706.02677*, 2017. URL <https://arxiv.org/abs/1706.02677>.

- Lin, T., Stich, S. U., Patel, K. K., and Jaggi, M. Don’t use large minibatches, use local SGD. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=Bley01BFPr>.

- Recht, B., Re, C., Wright, S., and Niu, F. Hogwild!: A lock-free approach to parallelizing stochastic gradient descent. In *Advances in Neural Information Processing Systems*, 2011.

- Stich, S. U. Local SGD converges fast and communicates little. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=S1g2JnRcFX>.

A. Additional Figures

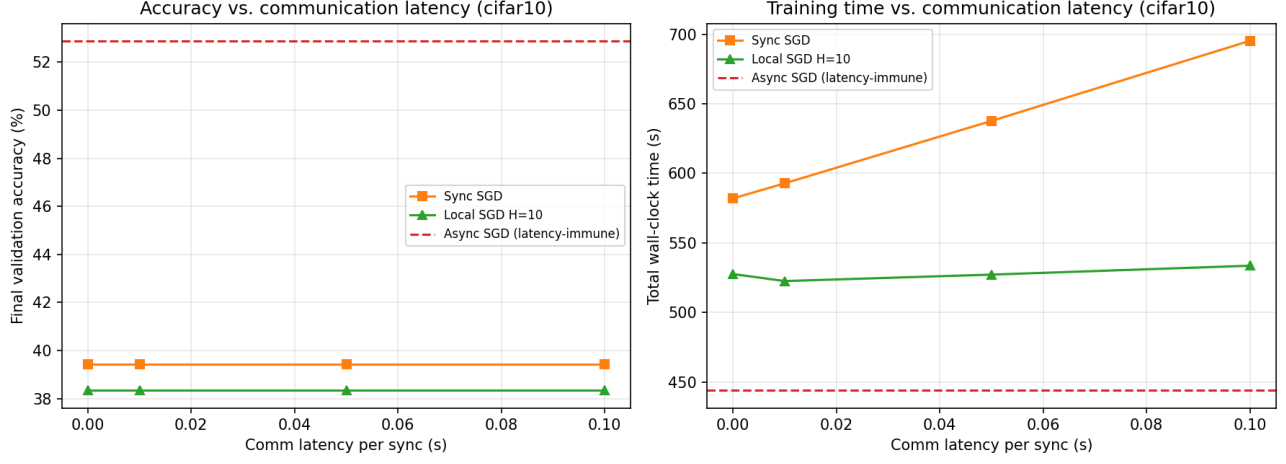


Figure 2. Effect of injected all-reduce latency t on wall-clock training time (left) and final validation accuracy (right) for synchronous SGD, Local SGD ($H=10$), and async SGD (4 workers, 5 epochs, CIFAR-10). Sync SGD overhead grows linearly with t ; Local SGD incurs approximately $10\times$ less overhead due to its reduced synchronization frequency; async SGD is unaffected as it performs no all-reduce.

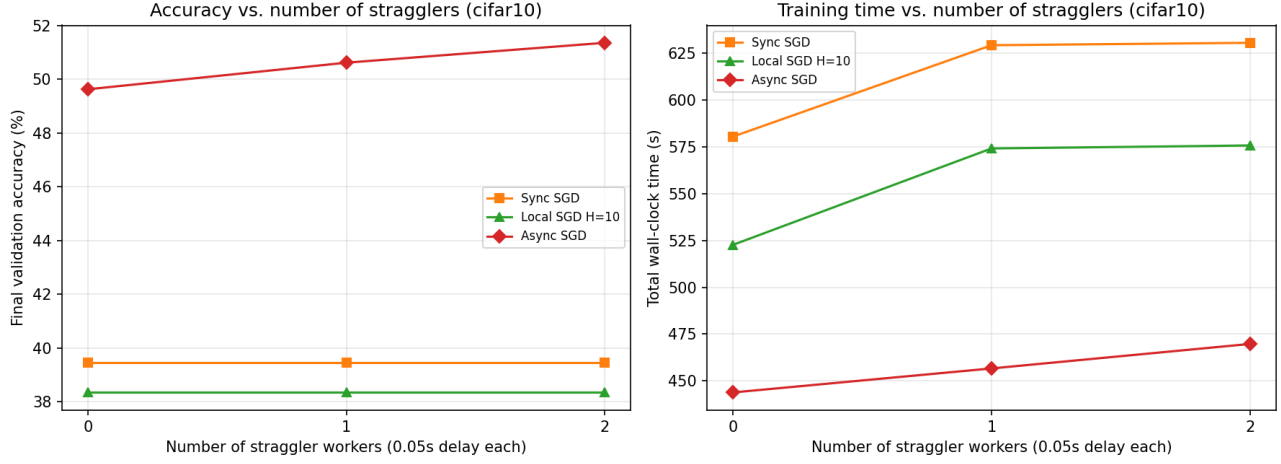


Figure 3. Effect of straggler workers on wall-clock training time (left) and final validation accuracy (right) for synchronous SGD, Local SGD ($H=10$), and async SGD (4 workers, 0.05 s straggler delay, 5 epochs, CIFAR-10). Synchronous and Local SGD incur equivalent overhead because straggler cost is determined by total computation, not synchronization frequency. Async SGD degrades gracefully: slow workers reduce update throughput but do not block the server.

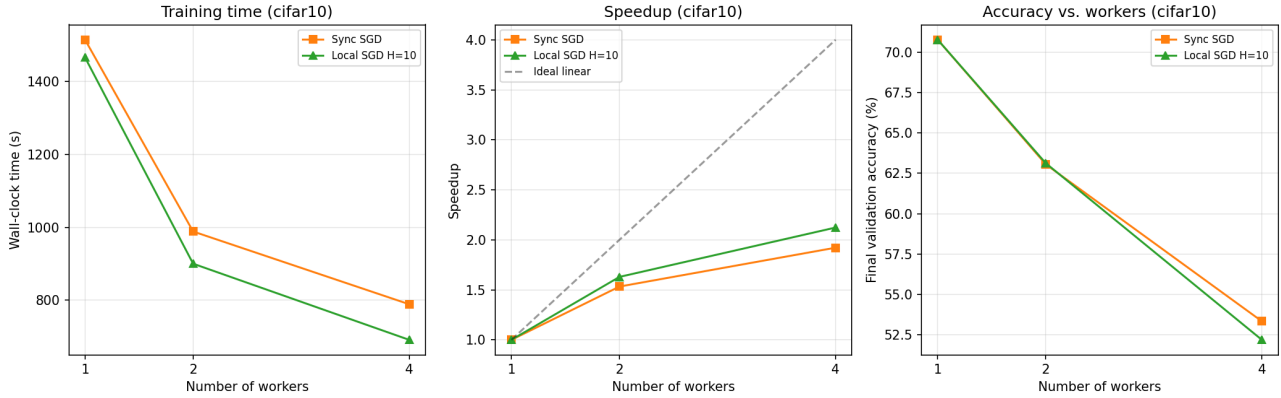


Figure 4. Worker scaling experiment for synchronous SGD and Local SGD ($H=10$) over 15 epochs on CIFAR-10. Left: wall-clock training time decreases sub-linearly with worker count. Center: parallel speedup T_1/T_W relative to the single-worker baseline. Right: scaling efficiency (speedup/ W), where 1.0 is perfect linear scaling. Local SGD achieves higher efficiency at every worker count due to its lower communication overhead.